

AI-Assisted Legacy App Migration Playbook

Oct 3, 2026 · ERM Slate to React migration

Summary

We moved the ERM Risk & Opportunity Dashboard from Foundry Slate to React and OSDK with no documentation to start from. An AI-assisted method made that possible, and this playbook makes it repeatable for any other legacy app.

The method has seven steps. It starts with packing the whole application into AI-ready context with our `pack-project.ts` script, and ends with architecture and lifecycle diagrams that document the new app. On ERM it produced a catalogue of 105 legacy features, a feature-by-feature comparison, targeted fixes and new features. We finished within the short timeline we set ourselves.

The challenge

The legacy app was large, spread across many files, and documented nowhere. Its behaviour lived only in Slate configuration and in the heads of its users.

Aspect	Legacy Slate app
Structure	1 shell app + 7 modules: print preview, action tracker, full search, heatmap, one-pager, report table, waterfall
Source size	8 Slate JSON exports, about 25 MB of widgets, variables, events, queries and embedded iframe code
Documentation	None: no specs, no diagrams, no feature list
Data and security	ARM risk data behind restricted views; Foundry functions and actions run as the user
Timeline	Short, set by the team itself

On top of parity, we wanted the React app to add new features rather than just copy the old one.

Step 0: Build AI context with pack-project.ts

An AI model can only reason about what it is given. `pack-project.ts` is a small script we wrote that turns a whole codebase into one file the model can read in one go.

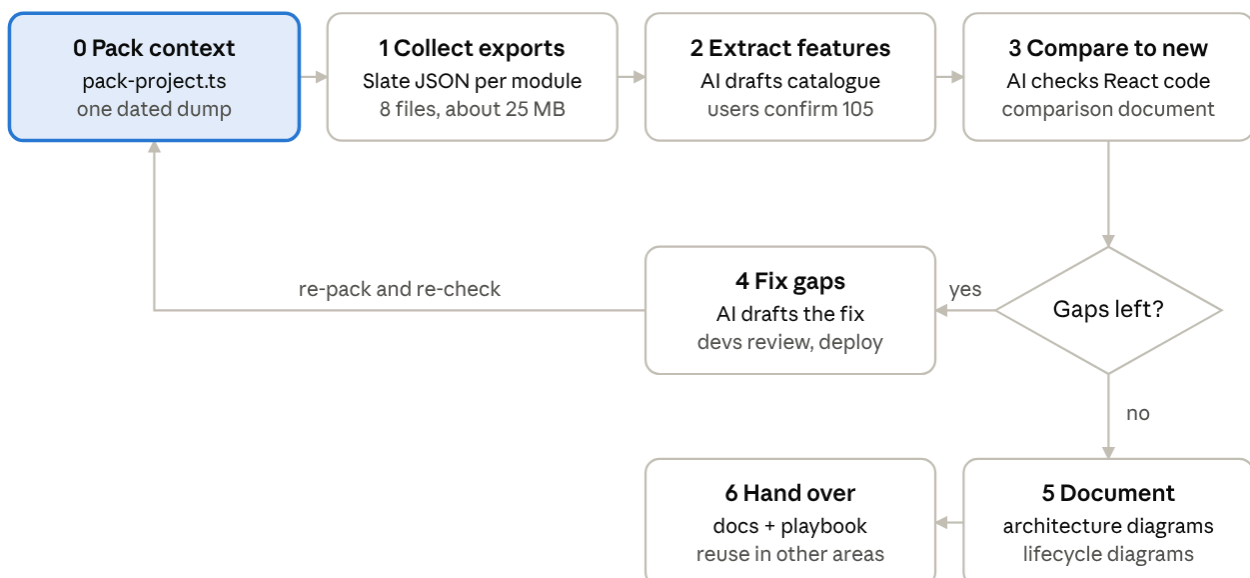
- **What it does:** walks the project folder and writes every source file into a single `PROJECT FILE DUMP`. Each file sits under a `FILE PATH:` header, and the dump is stamped with the time it was generated.
- **ERM example:** the React app packed into one text file of 75 source files, about 0.8 MB.
- **Why it matters:** the model sees the components, hooks, OSDK calls and types together, so its answers trace real code paths instead of guessing from fragments. Re-running the script after every change gives a fresh, dated snapshot, so the model never works from stale code.
- **Legacy side:** the Slate apps needed no packing, because each Slate export is already one self-contained JSON file per module.

The packed React dump plus the eight Slate exports are the complete context for every later step.

The playbook

Seven steps take a legacy app from undocumented to migrated and documented; steps 3 and 4 repeat until no unplanned gaps remain.

Fix and re-check until no gaps are left, then document



migration playbook · 7 steps, 1 loop

Each fix is re-packed and re-checked, so the comparison document always reflects the current code.

Step	Input	Output	Who checks
0 Pack context	New codebase	One dated dump from <code>pack-project.ts</code>	Developers
1 Collect exports	Legacy app	One JSON export per Slate module	Developers
2 Extract features	Exports	Feature catalogue (105 for ERM)	Business users
3 Compare to new	Catalogue + dump	Comparison document: parity, new features, gaps	Business users + developers
4 Fix gaps	Each gap	Reviewed, tested, deployed fix	Developers + function owners
5 Document	Final dump + exports	Architecture and lifecycle diagrams	Whole team
6 Hand over	All deliverables	Documented app; playbook for the next team	Team lead

Deliverables produced for ERM

Every step left a reusable artifact behind; together they form the documentation the app never had.

Deliverable	What it gives the team	Step
Packed React context	One dated file of the whole React codebase for AI review	0
Feature catalogue	All 105 legacy features, grouped by module	2
Feature comparison document (PDF)	Each legacy feature against React, plus React's new features and any gaps	3
Backend fix to <code>executeAddEditWorkflow</code>	Board members get access to risks when a dashboard is created or edited (deployed)	4
React fixes	App-level toast notifications; dashboard list refresh with retries	4

Deliverable	What it gives the team	Step
Architecture diagrams	Detailed views of the Slate app and the React app	5
Lifecycle diagrams	Dashboard lifecycle (Slate, React) and risk lifecycle (combined, React only)	5

Teamwork and cross-functional cooperation

With no documentation, no single group could rebuild the app alone; the playbook gave each group a clear part to play.

- **Business and developers rebuilt the knowledge together.** The AI drafted the 105-feature catalogue; [ERM business users / risk officers] confirmed it against how they really use the tool. The comparison document became the shared answer to "does the new app do X?".
- **Data and security owners were involved early.** ARM risk data sits behind restricted views. With [ARM data owners / platform team] we kept that model and fixed how access is granted when a dashboard is created or edited.
- **Backend and frontend moved together.** Function and UI fixes were reviewed and deployed together with [backend function owners].
- **Diagrams gave everyone one picture.** Business, development and support now describe the system the same way.

Open: name the teams and divisions involved and how many people took part.

Sustainability and transfer to other business areas

The playbook needs only the legacy app's export, the new codebase and an AI model, so it carries over to any team with a legacy app to replace.

Sustainable

- The app is now standard React and OSDK code: version-controlled, typed against the data model, easier to maintain than Slate configuration.
- An undocumented app now has a feature catalogue, a comparison document and architecture and lifecycle diagrams, so new joiners and support staff can learn it quickly.
- The security model (restricted views, functions that run as the user) was kept, so no governance rework was needed.

- `pack-project.ts` keeps the AI context current: re-pack after each change and re-run the review.

Transferable

- Other Slate apps in [other business areas] can follow the same seven steps unchanged.
- Other low-code or legacy tools with little documentation fit too: only the export in step 1 changes.
- `pack-project.ts`, the comparison document layout and the diagram formats are reusable as they are.

Jury demo script (about 10 minutes)

Show the method working, not just the result: each segment has one thing on screen.

Time	Show on screen	Say
0:00–1:00	The old Slate app and its folder of JSON exports	"Seven modules, 25 MB of configuration, zero documentation."
1:00–2:30	Run <code>pack-project.ts</code> live; open the dump	"One command gives the AI the whole app, dated and complete."
2:30–4:00	The 105-feature catalogue	"Business users checked this list; it replaced manual discovery."
4:00–5:30	The comparison document, one gap highlighted	"Every feature is tracked, so nothing is lost by accident."
5:30–7:00	One fix, before and after (e.g. dashboard list refresh with toast)	"Gaps turn into small, reviewed fixes."
7:00–8:30	Slate vs React architecture diagram; the risk lifecycle	"The app now explains itself to the next team."
8:30–10:00	The React app's new features, then the 7-step flow	"Any team can run this on their legacy app."

Before the demo:

- ☐ Pre-run `pack-project.ts` once in case the live run is slow
- ☐ Open the comparison PDF at the chosen gap
- ☐ Open the diagrams in the browser, one tab each
- ☐ Pick which new features to show

Lessons learned and guardrails

The AI speeds up every step, but people still own the decisions and the checks.

- **Complete context beats clever prompts.** Answers improved most once the model had the full packed codebase and every Slate module, not excerpts.
- **Humans confirm, AI drafts.** Business users signed off the feature catalogue; developers reviewed and tested every fix before deploying.
- **Declare deliberate differences.** Features left out by choice (the heatmap slider, report KPIs) are marked as such, so they are not reported as gaps.
- **Respect the data model's security.** Restricted views and run-as-user functions stay in place; fixes are checked against who can see which risk.
- **Use approved tools and channels.** Share code and data with AI only through tools and routes the company has approved.
- **Re-pack before each review.** A stale dump produces stale advice.